



R.B.INSTITUTE OF MANAGEMENT STUDIES

Master of Computer Applications

Subject Code: MC03094021

Subject Name: Software Engineering

Unit -2 Requirements engineering

Requirements Engineering

Requirements Engineering (RE) is the systematic process of identifying, analysing, documenting, validating, and managing the requirements of a software system. In simple words, it means finding out what the customer needs and clearly defining what the software should do.

Main Activities of Requirements Engineering

1. Requirement Elicitation

- Collect requirements from customers, users, and stakeholders.
- Techniques include interviews, questionnaires, observation, and meetings.

2. Requirement Analysis

- Study the collected requirements.
- Identify conflicts, missing information, duplication, and feasibility issues.

3. Requirement Specification

- Document the requirements clearly in a Software Requirements Specification (SRS) document.
- It describes what the system should do and its constraints.

4. Requirement Validation

- Correct
- Complete
- Consistent
- Feasible
- Unambiguous
- Testable

5. Requirement Management

- Manage changes to requirements throughout the software development process.
- Track versions and analyse the impact of changes.

Types of Requirements

- **Functional Requirements:** Describe **what the system should do**.
Example: "The system shall allow users to log in using email and password."
- **Non-functional Requirements:** Describe **how well the system should work**.
Example: "The webpage should load within 2 seconds."

1. Functional Requirements

Functional requirements describe what the system should do. They specify the features and functions of the software.

Examples:

- User can register and log in.
- User can search for products.
- Admin can add, edit, and delete records.
- System generates monthly reports.
- User can make online payments.

Key points:

- Define system behaviour.
- Describe inputs, processing, and outputs.
- Focus on business functions.

2. Non-Functional Requirements

Non-functional requirements describe how the system should perform. They define the quality, performance, and constraints of the system.

Examples:

- The system should load within 2 seconds.
- The application should be available 99.9% of the time.
- User passwords must be encrypted.
- The system should support 10,000 concurrent users.

- The interface should be easy to use and responsive.

Key points:

- Define quality attributes.
- Focus on performance, security, reliability, scalability, and usability.
- Improve the user experience.

Difference between Functional and Non-Functional Requirements

Functional Requirements	Non-Functional Requirements
Describe what the system does	Describe how the system performs
Define features and functions	Define quality and performance
Example: User login	Example: Login should complete within 2 seconds
Based on business needs	Based on system quality attributes

Requirements Engineering Process

Requirements Engineering (RE) is the systematic process of **identifying, analysing, documenting, validating, and managing** the requirements of a software system.

The main processes are:

1. **Feasibility Study**
 - Checks whether the proposed system is practical and possible to develop.
 - Considers cost, technology, time, and resources.
 - Example: Can the company afford to develop an online shopping system?
2. **Requirements Elicitation**
 - Collects requirements from customers, users, managers, and other stakeholders.
 - Techniques include interviews, questionnaires, observation, surveys, and meetings.
 - Example: Asking customers what features they want in a mobile banking app.

3. **Requirements Analysis**

- Examines and organizes the collected requirements.
- Identifies conflicts, missing requirements, duplicates, and unclear requirements.
- Requirements may also be prioritized.

4. **Requirements Specification**

- Requirements are written clearly in a formal document called the Software Requirements Specification (SRS).
- It describes functional and non-functional requirements.
- Example: "The system shall allow users to reset their password using their registered email."

5. **Requirements Validation**

- Checks whether the requirements are correct, complete, consistent, realistic, and testable.
- Stakeholders review the requirements to find errors before development begins.

6. **Requirements Management**

- Handles changes to requirements throughout the software project.
- Requirements can change because of new business needs, customer feedback, or technical limitations.
- Example: Adding UPI payment support after development has started.

Simple Flow

Feasibility Study → Elicitation → Analysis → Specification → Validation → Management

Examples of Requirements Engineering Tools

- **IBM DOORS** – Requirements management and traceability.
- **Jira** – Managing user stories, tasks, and changing requirements.
- **Azure DevOps** – Managing requirements, work items, development, and testing.
- **ReqView** – Requirements documentation and traceability.
- **Enterprise Architect** – Requirements modelling using UML and other diagrams.
- **Jama Connect** – Requirements, risk, traceability, and collaboration management.

Requirement Elicitation

Requirement elicitation is the process of understanding, collecting, analyzing, and documenting what the customer or users need from a software system or project.

- Understand the customer's actual needs
- Reduce misunderstandings
- Avoid costly changes later
- Define the project scope
- Improve software quality
- Give developers clear instructions

Main activities

1. **Identify stakeholders**
 - Find out who will use, manage, or be affected by the system.
 - Example: customers, managers, employees, administrators.
2. **Gather requirements**

Use different techniques to understand their needs:

 - Interviews
 - Questionnaires/surveys
 - Meetings/workshops
 - Observation
 - Brainstorming
 - Document analysis
 - Prototyping
3. **Analyse requirements**
 - Remove unclear or conflicting requirements.
 - Check whether the requirements are realistic and feasible.
4. **Prioritize requirements**
 - Decide which requirements are most important.
 - For example: **Must have, Should have, and could have.**
5. **Document requirements**

- Write the requirements clearly so developers, testers, and customers have the same understanding.

6. **Validate requirements**

- Confirm with stakeholders that the requirements are correct, complete, and understandable.

Why is requirement elicitation important?

- Understand the customer's actual needs
- Reduce misunderstandings
- Avoid costly changes later
- Define the project scope
- Improve software quality
- Give developers clear instructions

Software Requirement Specification Document (SRS)

A common form is called Software Requirements Specification (SRS). It usually contains:



1. **Introduction** – Purpose and scope of the software.
2. **Overall Description** – General features and users.

3. **Functional Requirements** – Functions the system must provide.
4. **Non-Functional Requirements** – Performance, security, reliability, etc.
5. **User Requirements** – What users expect from the system.
6. **System Constraints** – Limitations or restrictions.
7. **Interfaces** – User interface, hardware, software, and communication interfaces.

SRS Structure (Software Requirements Specification)

A standard **SRS document** generally contains the following structure:

1. Introduction

- Purpose
- Scope of the Software
- Definitions, Acronyms and Abbreviations
- References
- Document Overview

2. Overall Description

- Product Perspective
- Product Functions
- User Classes and Characteristics
- Operating Environment
- Design and Implementation Constraints
- Assumptions and Dependencies

3. Specific Requirements

3.1 Functional Requirements

- What the system should do
- Features and functions of the software

3.2 Non-Functional Requirements

- Performance Requirements
- Security Requirements
- Reliability Requirements
- Usability Requirements
- Maintainability Requirements

3.3 External Interface Requirements

- User Interface
- Hardware Interface

- Software Interface
- Communication Interface

4. System Features / Modules

For each module:

- Feature Name
- Description
- Inputs
- Processing
- Outputs
- Functional Requirements

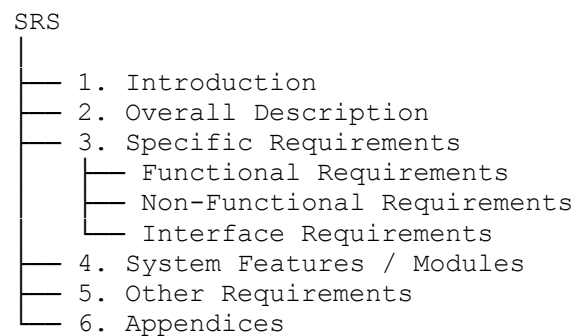
5. Other Requirements

- Database Requirements
- Legal Requirements
- Safety Requirements
- Business Rules

6. Appendices

- Glossary
- Analysis Models
- Supporting Information

Simple structure:



Requirement Validation

Requirement Validation is the process of checking whether the requirements written for a software system are correct, complete, consistent, realistic, and clear.



Why is requirement validation needed?

It helps to find errors in requirements at an early stage. Fixing a requirement error later can be expensive and time-consuming.

Main checks in Requirement Validation

1. **Validity**

- Checks whether the requirements actually represent the needs of the customer.
- Example: The customer needs online payment, so the requirement should include an online payment facility.

2. **Consistency**

- Requirements should not contradict each other.

Example:

- Requirement 2: User password must contain exactly 6 characters.
 - Requirement 1: User password must contain 8 characters.
 - These requirements are inconsistent.
3. **Completeness**
- All necessary requirements should be included.
 - Example: If a login system is required, requirements should cover login, password reset, and error handling where necessary.
4. **Realism / Feasibility**
- Checks whether the requirements can actually be implemented with available technology, time, and budget.
 - Example: A requirement asking for unlimited storage with a very small budget may not be realistic.
5. **Verifiability**
- Requirements should be written so that they can be tested.
 - Bad: "The system should be very fast."
 - Good: "The system should respond within 2 seconds."
6. **Clarity / Unambiguity**
- Requirements should have only one clear meaning.
 - Avoid vague words such as *fast*, *easy*, *user-friendly*, or *efficient* unless they are properly defined.

Requirements Change

Requirements change occurs when the customer or users want something different from what was originally specified.

Why do requirements change?

Requirements may change because of:

1. **Change in customer needs**
 - The customer may ask for new features.
2. **Change in business requirements**
 - Business rules or strategies may change.
3. **New technology**
 - New technology may make a different solution possible.

4. **Government or legal changes**

- New laws or regulations may require changes to the system.

5. **Errors in original requirements**

- A requirement may have been misunderstood or incorrectly specified.

6. **Market competition**

- Competitors may introduce new features, forcing changes.

Requirement Change Process

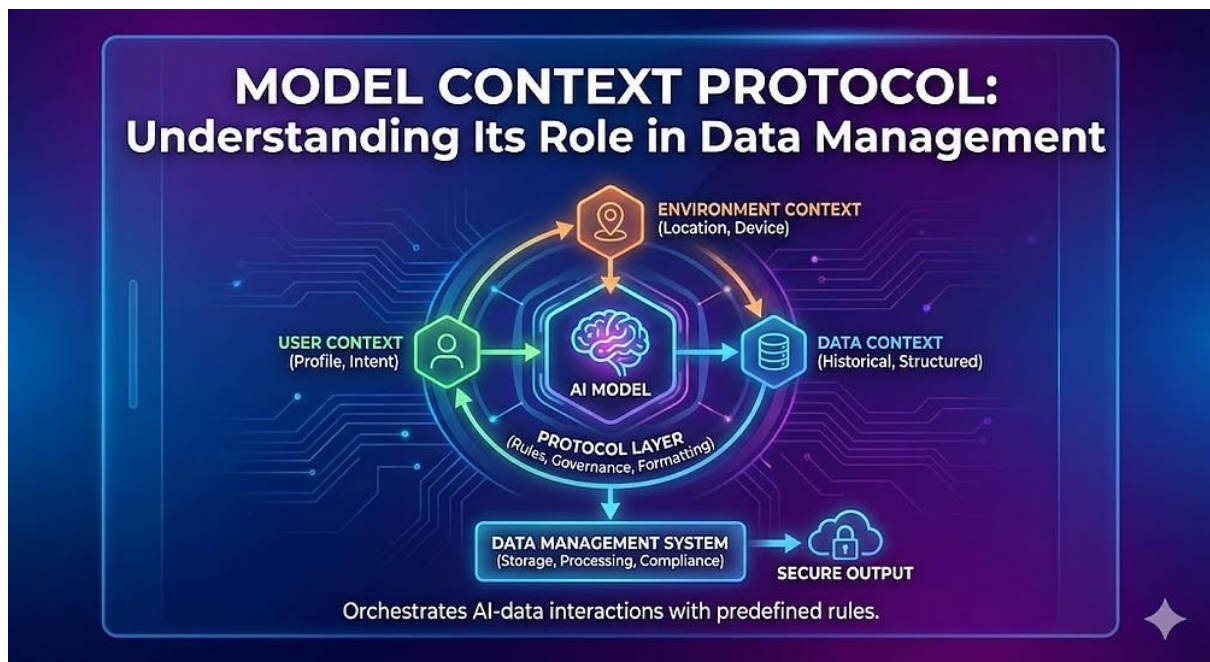
A typical process is:

Change Request → Impact Analysis → Approval → Update Requirements → Implement Change → Test

- **Change Request:** Someone requests a change.
- **Impact Analysis:** Developers determine how the change will affect cost, time, design, and other requirements.
- **Approval:** The change is accepted or rejected.
- **Update Requirements:** Requirement documents are updated.
- **Implementation:** Developers make the necessary changes.
- **Testing:** The modified system is tested.

System modelling: Context models

A context model is an external perspective that shows what is inside and outside a software system. It uses a high-level view to map out system boundaries, external environment entities, and interactions to clarify project scope and requirements.



Key Elements of Context Models

- **System Boundaries:** Lines that separate the target system from its surrounding environment and external components.
- **External Entities:** Boxes representing other software systems, hardware, organizations, or users that interact with the system.
- **Data and Control Flows:** Connectors showing the exchange of data, funds, or physical items across the boundary

Purpose and Use

- **Scope Definition:** Sets clear limits on what the system will and will not handle.
- **Communication:** Acts as a simple tool to explain system placement to non-technical stakeholders.
- **Requirements Engineering:** Helps discover initial functional needs by observing how the system fits into broader business workflows

1. Interaction Models

Interaction models describe **how users, objects, or systems interact with each other**.

They focus on the **communication and sequence of actions**.

Example: Online shopping:

1. Customer selects a product.
2. Customer adds it to cart.

3. Customer makes payment.
4. System confirms the order.

Common diagrams:

- **Sequence diagram** – shows the order in which interactions happen.
- **Use-case diagram** – shows how users interact with a system.

🔑 **Focus:** *Who interacts with whom, and in what order?*

2. Structural Models

Structural models describe the **static structure of a system**—what components exist and how they are related.

They don't focus on what happens over time.

Example: In a college management system:

- Student
- Teacher
- Course
- Department

A Student may enrol in a Course, and a Teacher may teach a Course.

Common diagrams:

- Class diagram
- Object diagram
- Component diagram
- Package diagram

🔑 **Focus:** *What is the system made of and how are its parts connected?*

3. Behavioural Models

Behavioural models describe how a system behaves or changes over time in response to events or inputs.

Example: ATM:

Insert Card → Enter PIN → Select Withdrawal → Enter Amount → Dispense Cash

The system changes its state depending on what the user does.

Common diagrams:

- State-machine diagram
- Activity diagram
- Sequence diagram

☞ **Focus:** *How does the system behave when something happens?*

4. Model-Driven Architecture (MDA)

Model-Driven Architecture (MDA) is an approach to software development where models are created first and then used to develop the actual software. The main idea is to separate the design of the system from the technology used to **implement it**.

For example:

Platform-Independent Model (PIM)



Describes the system without depending on a particular programming language or platform.

Platform-Specific Model (PSM)



Adapts the design to a specific technology, such as Java, .NET, or a particular database.

Code



Actual software is generated/implemented from the models.

Simple example

Suppose you're designing a banking application.

PIM:

"Customer can create an account and transfer money."

PSM:

"Implement Customer using Java classes and store account data in MySQL."

Implementation:

Actual Java + MySQL application.

☞ **Focus:** *Build software from models and make the design less dependent on a specific technology.*

Easy way to remember

Model	Main question
Interaction Model	How do things/users interact?
Structural Model	What things/components exist?
Behavioral Model	How does the system behave/change?
MDA	How can models guide development independently of technology?

User Requirement

User requirements are high-level statements, written in simple language, that describe the services and functions a system must provide and the constraints under which it must operate.

- **Easy & Simple to Operate**
- **Quick Response**
- **Effectively handling operational errors**
- **Customer Support**

User Requirements Specification

A User Requirements Specification is a formal document that systematically defines, describes, and organizes the users' needs, expectations, system services, and constraints.

It is contractual agreement.